THE HACKING PROJECT

Grilles d’évaluation du projet final

Session 6 - Présentation de El Podcast par Cédric Soares

Sommaire

Sommaire 1

Mode d'emploi de la grille 2

Jury Produit 3

Jury Technique 6

Jury Cursus 8

Annexes 10

Sites avec un front non recevable 10

Sites avec un front très limite 10

Sites qui passent très bien 10

Mode d’emploi de la grille

Un projet final est considéré comme valide si :

- Il respecte tous les critères éliminatoires (c’est à dire que vous n’avez fait aucune des erreur éliminatoire, comme indiquées dans chaque grille)

Des points bonus seront aussi attribués pour vous récompenser du travail supplémentaire. Par exemple, utiliser une méthode en peer review ravira vos examinateurs !

Vous trouverez ci-dessous le détail de chacune des grilles des différents jurys.

Jury Produit

✓ **Points éliminatoires**

1. La page d'accueil du site n'explique pas clairement ce que propose votre application (ex: pas de proposition de valeur, page d'accueil vide ou presque, trop de texte et ce n'est pas assez clair)
    a. Nous attendons que la page d'accueil explique bien l'application. Pour tester ceci simplement, allez voir une personne inconnue et demandez-lui ce qu'elle pense qu'est votre application à partir de la page d'accueil du site
2. La fonctionnalité principale de l'application n'est pas utilisable (ex: il y a les réponses en clair pour une application de quizz, cliquer sur un événement sur une plateforme d'événements affiche une erreur)
    a. Il est indispensable que votre application marche. Testez votre application, faites-la tester. Il faut qu'elle soit utilisable.
3. La navigation sur le site est compliquée (ex: la navbar n'est pas utilisable, il faut cliquer sur *retour* pour accéder aux différentes fonctionnalités, les liens pour naviguer sont difficiles à trouver, il n'y a pas de lien pour naviguer sur le site)
    a. Pouvoir naviguer facilement d'un point à l'autre de l'application est indispensable. Baladez-vous, faites tester une navigation à une personne inconnue pour voir si elle ne se perd pas
4. La page d'inscription (signup) sur le site demande autre chose qu'email et mot de passe (ex: demander une photo de profil à l'inscription, demander une date de naissance à l'inscription, demander une bio à l'inscription, etc), même si le champ n'est pas obligatoire. Même si ces champs ne sont pas obligatoires à remplir, le moindre formulaire de signup qui demande autre chose que email mot de passe, **toute l'équipe passe pirate**.
    a. Il est possible de demander par exemple prénom et nom, mais vous avez intérêt à avoir une justification valable et le jury vous demandera pourquoi votre signup demande cela.

- ✓ **User Experience (UX)**
  1. Intelligibilité
     a. On comprend instantanément ce que propose le site, dès la première connexion
     b. Le site est capable de suggérer sa propre utilisation (quand tu arrives sur le site, tu sais directement comment l'utiliser)
     c. Le site répond au problème proposé ?
  2. Utilisabilité
     a. Il n'y a pas de friction dans le parcours utilisateur ? (point de vue UX uniquement, donner un exemple)
     b. Tous les items ont une raison d'être / une fonction (pas de liens morts, pas de liens vides, etc...)
     c. L'utilisateur est guidé et la construction du site favorise une visite fluide. (Peu importe où l'utilisateur va sur le site, il ne se perd pas)

- ✓ **User Interface (UI)**
  1. Lisibilité
     a. Pas plus de trois polices
     b. Tailles de police adaptée à l'écran (pas trop petit ni trop gros)
     c. Cohérence dans les tailles de textes (ex: toujours la même taille pour h1, h2, etc..)
  2. Couleurs
     a. Pas plus de 5 couleurs
     b. Les couleurs choisies sont complémentaires et en harmonie chromatique (= vont bien ensemble) ?
     c. Le contraste fond/texte n'empêche pas la lisibilité
     d. Il y a une cohérence dans les couleurs de textes
  3. Organisation
     a. La position des éléments sur la page est cohérente ? (ex: la navbar est toujours au même endroit)
     b. Le site est aéré ? (présence d'espaces vides)
     c. Le site est sobre ? (pas de présence d'immenses pavés)
     d. Il y a un logo sur la page d'accueil (Voici un exemple de générateur)
     e. Il y a un slogan sur la page d'accueil
     f. La proposition de valeur est située en dessous du slogan.
  4. Photos/vidéos

    a. Les photos/vidéos sont de qualités (pas trop pixellisées) ?
    b. Les photos/vidéos n'empêchent pas le chargement du site ? (Un outil comme web.dev vous permet de mesurer le temps de chargement)
    c. Les photos/vidéos n'empêchent pas la visibilité
    d. BONUS : Les photos/vidéos ont un filtre qui facilite la lecture

✓ **Besoin**

1. Démarche de réflexion
    a. Le projet a-t-il une volonté de répondre à un problème ?
    b. BONUS : L'équipe est-elle en phase avec le problème qu'elle vise ? (expérience dans le milieu, connaissance sur le sujet, relations, etc...)
2. Répondre à un problème
    a. Le projet apporte-t-il une réponse (même partielle) au problème proposé ?

✓ **Proposition de valeur**

1. Définition de la proposition de valeur
    a. La proposition de valeur est-elle déclarée ?
    b. La proposition de valeur est-elle claire ?
    c. La proposition de valeur est-elle unique ? (pas plus d'une proposition de valeur)
2. Respect de la proposition de valeur
    a. La proposition de valeur est-elle réaliste ? (pas de promesse intenable)
    b. La proposition de valeur est-elle atteinte ? (les promesses du site sont respectées)

Jury Technique

- ✓ **Points éliminatoires**
  1. Il y a des mélanges de langages inappropriés (ex: du HTML dans les controllers, des message de conflits de git n'importe où)
     a. Relisez-vous bien, faites des pull requests, des code reviews pour éviter que du mauvais code s'insère dans l'application. Si nous voyons le moindre code inapproprié dans l'application, c'est toute l'équipe qui passe pirate
  2. Il y a plusieurs models sans relations entre eux
  3. L'architecture de l'application n'est pas RESTway
     a. La moindre route en POST, et c'est tout le monde qui passe pirate. Quelques GET sont autorisés pour les quelques pages statiques par exemple
  4. Les clefs d'API sont en clair dans le code
  5. Les noms de classes, de models et de variables sont en français
     a. Le moindre mot en français, et toute l'équipe passe pirate. Relisez-vous bien, faites des pull requests et code reviews et évitez un fail à cause d'un "oubli"
  6. Les liens sont hardcodés et n'utilisent pas les helpers (<a href="www.tonsite.com/users/23"></a> et <img src="imgur.com/photo.jpg"/>)
- ✓ **Backend**
  1. <u>Le code</u>
     a. Fat Model Skinny Controllers
     b. Utilisation de services pour leur but originel (alléger les controllers)
     c. Business Logic : le nom des tables est orienté produit (les tables ont un sens, exemple "réservation")
     d. Présence d'un mailer (dans le dossier app/mailers)
     e. Routes propres et utilisation de nested resources quand il faut
     f. Il y a des validations dans les models (pour empêcher de saisir des données incorrectes)
     g. Les APIs sont enveloppées dans des services
  2. <u>Base de donnée</u>
     a. Les bon types d'attributs correspondent aux bons paramètres (string pour zipcode par ex)
     b. Les migrations sont propres et on peut les répliquer sur notre machine
     c. Présence d'un Seed

d. Les relations N-N sont en has_many through (et non pas has_and_belongs_to_many)

✓ **Frontend**

a. Le site est-il responsive ?
b. Le code est DRY et l'app utilise autant que possible des partials ?
c. L'app utilise autant que possible des composants ?
d. Framework CSS
e. Utilisation des rails helpers (link_to, image, fichiers helpers, etc...)
f. Tous les assets sont bien présents en production ? (pas de carré vide pour les icônes, images non chargées)
g. Utilisation de hotwire
h. Utilisation des assets (code bien rangé dans les assets)

Jury Cursus

✓ **Backend**

1. Points éliminatoires

a. Pas d'application fonctionnelle en production
b. Pas de système d'authentification branché (il faut les 5 views inscription, connexion, édition, mot de passe oublié, réinitialisation du mot de passe, etc)
c. Il n'y a pas de mailer fonctionnel en production
d. Pas d'API externe. Il faut au moins UNE API (hors mailer et google maps)
e. Il faut au moins deux models (dont les users), reliés entre eux par des associations
f. Il n'y a pas de front (celui-ci doit impérativement être orienté composants)
g. Les 5 vues de devises n'ont pas de css
h. Le front est irrecevable

        i. **Le site est illisible**
    2. Concepts clefs
        a. Il y a un dashboard admin
        b. Utilisation de Hotwire et Stimulus

✓ **Travail en équipe**

1. Points éliminatoires
    a. **Il y a des personnes qui n'ont pas ou peu codés (on ira vérifier les commits)**
    b. **Il faut un trello pour la gestion de projet (ou Asana / Atlassian)**
    c. **Le code n'est pas hébergé sur Github ou Gitlab**
    d. **Il faut utiliser la fonctionnalité de branches de Git et Github**
2. Bonnes pratiques
    a. Le code est-il commenté ?
    b. Le code est correctement indenté ?
    c. Pas de commit sur Master
    d. Les commits sont fréquents ? (Pas de commit de 400 lignes de codes)
    e. Les messages de commit sont explicites ?
    f. Les messages de commit sont en anglais ?
    g. Y a-t-il un pipeline de production ? (app de développement et app de production)
    h. Y a-t-il une branche par fonctionnalité ?
3. Travail en équipe
    a. Des concepts d'agilité ont été utilisés dans la gestion de projet
    b. Stand-up quotidiens
    c. Une répartition équitable du travail (vérifiée au nombre de commits par personnes)